



Project Retrospective – Team 4 – TaskOrbit

What worked?

Overall, the project benefited from strong teamwork and smooth communication between the developers, Scrum Master, and industry partner. Questions and problems could usually be resolved quickly through Discord or regular meetings, which prevented unnecessary delays. Within the development team, responsibilities were distributed efficiently, particularly during internal meetings, and team members were willing to take ownership of tasks.

The additional meeting formats also supported the development process. The Dev–PO meetings provided a useful setting for sprint planning and the validation of implemented features. The internal developer meetings helped the team assign tasks, discuss technical challenges, and coordinate ongoing work. The meetings with the industry partner ensured regular alignment with the project requirements and expectations. Together, these meetings improved coordination and provided clear opportunities to address issues throughout the project.

The team maintained a good level of code quality. The Git branching strategy, pull requests, and automated GitHub Actions checks (CI/CD Pipeline) helped identify issues early and ensured consistency. Pair programming was especially effective for complex features that were difficult to divide into separate tasks. It accelerated development, supported knowledge sharing, and made the work more enjoyable. Overall, the project achieved its main objectives and provided valuable technical and teamwork experience.

What didn't work?

The main challenge was the increasing number of bugs and compatibility issues across multiple sprints. Features that had worked during previous sprint reviews sometimes stopped functioning after later changes to shared parts of the code. The agent handoff feature was a particularly significant example, as repeated modifications eventually caused it to stop working completely. The ongoing and worsening bug situation created considerable stress within the team.

One reason these problems were identified too late was the lack of sufficient end-to-end testing. Although individual changes were reviewed and checked, the complete product was not tested thoroughly enough across all components and user flows. Some external limitations also made comprehensive testing more difficult. Consequently, several bugs only became visible late in the project instead of being detected shortly after the relevant changes were introduced.

Another major issue was that the deadlines for pull requests were rarely respected. Although the release manager was expected to reject pull requests submitted after the deadline, this rule was not consistently enforced. As a result, changes were often integrated too late, forcing developers to resolve merge conflicts under severe time pressure and, in some cases, work late into the night. The developers' motivation and willingness to invest this additional effort were highly commendable. However, this approach likely created more problems than it solved, as rushed conflict resolution and insufficient testing increased the risk of further defects. A stricter enforcement of pull request deadlines and a clear rejection of late submissions would have created a more stable and predictable integration process.

The team also continued adding new features until almost the final sprint. This left too little time to test the overall product, resolve accumulated bugs, and ensure that existing features remained compatible.